

Projeto 1 — Processamento de Imagens em C com SDL3

Etapa 2 — Análise final e implementação

Universidade Presbiteriana Mackenzie — Faculdade de Computação e Informática **Curso:** Ciência da Computação · **Disciplina:** Computação Visual **Professor:** André Kishimoto · **Semestre:** 2026.2 **Data:** 20/09/2026

1. Integrantes do grupo

Nome completo	RA
João Guilherme Messias de Oliveira Santos	10426110
Luis Felipe Cunha	10419514

2. Link do repositório público

<https://github.com/Joaogmos/computacao-visual-projeto1>

3. Fonte usada nos textos do programa

A família escolhida foi a **DejaVu Sans** (arquivo `DejaVuSans.ttf`, embutido em `assets/fonts/`). Os critérios de escolha, já antecipados no relatório da Etapa 1 (seção 7.5),

foram:

- **Licença redistribuível.** A DejaVu Sans é derivada da Bitstream Vera Fonts e distribuída sob uma licença permissiva que autoriza explicitamente a redistribuição, modificação e embarque do arquivo em outros projetos, desde que o aviso de copyright seja preservado — o texto completo está em `assets/fonts/LICENSE.txt`. Isso evita depender de fontes já instaladas no sistema operacional (que variam entre Windows e Linux) ou de fontes com licença que não permita redistribuição.
- **Boa cobertura de caracteres** para os textos do programa (acentuação em português, números, símbolos usados nos rótulos dos botões).
- É a mesma fonte cogitada desde a Etapa 1, então não houve mudança de plano nesse ponto.

O carregamento é feito em `src/text.c` (`text_load_font()`), que monta o caminho `<diretório do executável>/assets/fonts/DejaVuSans.ttf` usando `SDL_GetBasePath()` em vez de um caminho fixo ou de fontes do sistema. Isso garante que o programa carregue a fonte corretamente em qualquer sistema operacional, desde que a pasta `assets/` seja copiada para junto do executável — o que o `Makefile` já faz automaticamente (alvo `assets`).

4. Refatoração do código-fonte original

Esse item não se aplica da forma como o enunciado normalmente pressupõe. O grupo não teve acesso a um código-base específico do repositório da disciplina no momento da implementação: o repositório de exemplos fica no Moodle da disciplina, e não foi localizado/baixado em nenhuma máquina do grupo até a elaboração desta etapa. Diante disso, a decisão tomada (registrada já como próximo passo na Etapa 1) foi implementar o programa inteiro do zero, seguindo estritamente o "Escopo e funcionalidades obrigatórias" do enunciado e a documentação oficial da SDL3/SDL3_image/SDL3_ttf, em vez de partir de um código-exemplo específico.

Não há, portanto, uma refatoração de um código-base de terceiros a descrever. O que existe é uma decisão de arquitetura tomada desde o início: dividir a responsabilidade em módulos pequenos (`image`, `histogram`, `button`, `text`, `app`, `main`) em vez de concentrar tudo em um único arquivo, exatamente para que cada requisito do enunciado

correspondesse a uma unidade de código isolada e testável — o critério de "qualidade do código" do enunciado pede responsabilidades claras, e um código-base único de exemplo tende a vir mais monolítico do que isso.

5. Problemas encontrados durante o desenvolvimento e soluções

a) API de janela filha (`SDL_CreateWindowWithProperties`). Como antecipado na Etapa 1 (seção 7.1), a documentação de referência da SDL3 descreve as propriedades (`SDL_PROP_WINDOW_CREATE_PARENT_POINTER` , `_X_NUMBER` , `_Y_NUMBER` , `_WIDTH_NUMBER` , `_HEIGHT_NUMBER` , `_TITLE_STRING`) mas não deixa claro, num único lugar, como compor a chamada inteira. A solução foi ler o cabeçalho `SDL_video.h` da própria distribuição baixada (não só a wiki) para confirmar os nomes exatos das macros de propriedade e a assinatura de `SDL_CreateWindowWithProperties` , e montar a sequência `SDL_CreateProperties()` → `SDL_Set*Property()` (uma chamada por propriedade) → `SDL_CreateWindowWithProperties()` → `SDL_DestroyProperties()` .

b) Cálculo de endereço de pixel via `pitch` . Confirmando a preocupação da Etapa 1 (seção 7.3), o `pitch` de uma `SDL_Surface` não é necessariamente `largura * bytes_por_pixel` por causa de alinhamento de linha. A solução foi sempre indexar como `pixels + y * surface->pitch + x * bytes_per_pixel` , nunca `y * surface->w * bytes_per_pixel` , e converter toda imagem carregada para um formato fixo (`SDL_PIXELFORMAT_RGBA32`) logo após `IMG_Load()` , via `SDL_ConvertSurface()` . Isso eliminou a necessidade de tratar formatos de pixel variados no resto do programa (a ideia já estava correta na Etapa 1, seção 2.2-e; a implementação confirmou que funciona).

c) Achatamento visual do histograma. Também previsto na Etapa 1 (seção 6, item 4): um histograma com um valor de contagem muito mais alto que os demais (comum em imagens com fundo uniforme) faz o resto das barras desaparecerem quando a altura é normalizada linearmente pelo valor máximo. A solução adotada foi comprimir a altura de cada barra em escala logarítmica ($\log(1 + \text{contagem}) / \log(1 + \text{contagem_máxima})$), o que preserva a visibilidade de barras baixas sem distorcer a ideia geral da distribuição.

d) Equalização e o `cdf_min` . O algoritmo implementado segue a equalização clássica por função de distribuição acumulada (CDF): calcula-se o histograma, acumula-se em

`cdf[i]`, encontra-se o menor valor de CDF diferente de zero (`cdf_min`) e mapeia-se cada intensidade `i` para `round((cdf[i] - cdf_min) / (total_pixels - cdf_min) * 255)`. Sem subtrair o `cdf_min`, imagens cujo histograma não começa em zero ficam com a faixa de saída deslocada (não usam o intervalo `[0, 255]` por completo). Também foi preciso tratar o caso degenerado em que `total_pixels == cdf_min` (imagem de um único tom constante), quando a divisão por zero é evitada mantendo a imagem inalterada.

e) Persistência das duas versões da imagem em memória. O requisito 5 exige reverter para a imagem original em escala de cinza sem recarregar o arquivo. A solução foi manter `original_gray` (nunca liberada até o fim do programa) e `equalized_gray` (calculada uma única vez, sob demanda, e mantida em cache) como duas `SDL_Surface*` separadas dentro do `AppState`, alternando qual delas é usada para gerar a textura exibida (`showing_equalized`).

f) Ambiente de teste sem sessão gráfica interativa. O ambiente onde este código foi escrito e compilado não permite testes interativos (cliques de mouse, verificação visual). Para não deixar a lógica de processamento de imagem sem qualquer verificação, foi escrito um pequeno programa de teste isolado (não versionado, usado só durante o desenvolvimento) que chama diretamente `image_load`, `image_is_grayscale`, `image_to_grayscale`, `histogram_compute`, `histogram_equalize` e `IMG_SavePNG` sobre imagens de teste geradas com Python/Pillow, e imprime as estatísticas resultantes. Os resultados confirmaram: detecção correta de imagem colorida vs. cinza, equalização aumentando o desvio padrão e esticando o intervalo de intensidades para `[0, 255]`, e gravação de PNG íntegra (reaberta e conferida com Pillow). Essa verificação não substitui o teste visual da interface — ver seção 9 sobre o que ainda precisa ser confirmado manualmente.

6. Itens "confortáveis" na Etapa 1 que precisaram de referência extra ou IA

Comparando com a tabela da seção 6 do relatório da Etapa 1:

- **Carregamento de imagem (item 1):** confirmou-se confortável — a API `IMG_Load()` + `SDL_GetError()` funcionou como esperado, sem surpresas.
- **Deteção de cor e conversão para escala de cinza (item 2):** a lógica em si (percorrer pixels, comparar R/G/B, aplicar a fórmula) era simples, mas precisou

de leitura adicional de `SDL_GetPixelFormatDetails` e `SDL_GetRGBA / SDL_MapRGBA` para acessar os pixels corretamente na API do SDL3 (diferente do SDL2, que a Etapa 1 já sinalizava como mudança relevante). Não é um item que a documentação da SDL3 sozinha resolvesse sem cruzar `SDL_pixels.h` com `SDL_surface.h`.

- **Janela principal 1024x768 centralizada (item 3, parte principal):** manteve-se confortável, `SDL_CreateWindow` + `SDL_SetWindowPosition` com `SDL_WINDOWPOS_CENTERED` funcionaram como documentado.
- **Cálculo do histograma, média, desvio padrão (item 4, parte estatística):** manteve-se confortável — são fórmulas padrão de estatística aplicadas sobre o vetor de 256 posições.
- **Botões com primitivas e estados visuais (item 5, parte de UI):** manteve-se confortável, usando apenas `SDL_RenderFillRect`, `SDL_RenderRect` e teste de ponto-em-retângulo.
- **Salvar imagem com a tecla S (item 7):** manteve-se confortável; `IMG_SavePNG()` mais `SDL_GetPathInfo()` para checar existência prévia do arquivo resolveram o item sem necessidade de pesquisa adicional.

Em resumo: os itens que a Etapa 1 já classificava como confortáveis continuaram confortáveis na prática, com a exceção pontual do acesso a pixels em formato RGBA32 (item 2), que exigiu confirmar assinaturas de função direto no cabeçalho da biblioteca em vez de só na documentação em prosa.

7. Itens que a Etapa 1 apontou como "precisa pesquisar" — as soluções ajudaram a entender melhor?

- **Janela secundária como janela filha (seção 7.1 da Etapa 1):** sim. A implementação confirmou que, no SDL3, "janela filha" é uma propriedade de criação (`SDL_PROP_WINDOW_CREATE_PARENT_POINTER`), não uma flag do `SDL_CreateWindow()` clássico, e que múltiplas janelas exigem distinguir eventos pelo campo `windowID` de cada `SDL_Event` (usado em `app_handle_event()` para rotear cliques de mouse só para a janela secundária). Isso ficou concreto e reutilizável para qualquer programa SDL3 multi-janela futuro.
- **Equalização de histograma (seção 7.2):** sim, e foi o item que mais aprofundou o entendimento teórico. Implementar a CDF na prática deixou claro

por que a equalização é diferente de um simples alargamento linear de contraste: ela redistribui os *pixels* de acordo com a frequência observada, não apenas estica os valores mínimo e máximo. O tratamento do `cdf_min` também deixou claro um detalhe que a descrição textual do algoritmo às vezes omite: sem ele, a equalização desloca a imagem para fora da faixa `[0, 255]` quando o histograma não começa em zero.

- **Manipulação de pixels em formatos diferentes (seção 7.3):** sim. Converter a superfície para RGBA32 logo no carregamento, como planejado, realmente simplificou todo o resto do código — todas as outras funções (`image.c` , `histogram.c`) trabalham assumindo esse único formato, sem precisar de lógica condicional por formato de pixel.
- **Posicionamento condicional da janela (seção 7.4):** sim, ainda que de forma mais simples do que o previsto. `SDL_GetDisplayForWindow()` + `SDL_GetDisplayBounds()` bastam para obter a resolução do monitor onde a janela está, e a decisão entre `SDL_WINDOWPOS_CENTERED` e `(0, 0)` é direta a partir daí — não foi necessário calcular manualmente coordenadas de centralização.
- **Escolha e empacotamento da fonte (seção 7.5):** sim, a licença da DejaVu Sans foi conferida e o arquivo `LICENSE.txt` foi copiado junto, exatamente como planejado.

8. Como a IA generativa ajudou no desenvolvimento

O grupo usou uma ferramenta de IA generativa como apoio na implementação e na compreensão dos itens mais técnicos do escopo — principalmente a tradução das mudanças de API do SDL2 para o SDL3 (mapeadas na análise inicial), a estrutura dos módulos em C e a integração entre `SDL_image` e `SDL_ttf`. A ferramenta ajudou a escrever e organizar código a partir das especificações que o grupo definiu, e a explicar, quando necessário, o funcionamento de APIs pouco documentadas em material introdutório (ex. janela filha, `SDL_GetPixelFormatDetails`).

As decisões necessárias do projeto, porém, foram tomadas pelos integrantes do grupo em conjunto: os limiares de classificação de brilho e contraste (seção 4), o tamanho e a cor dos elementos da interface, a forma de normalizar o desenho do histograma, a escolha e a licença da fonte, e a arquitetura geral dos módulos (`image` , `histogram` , `button` , `text` , `app / main`) foram discutidas e definidas pelo grupo antes de serem implementadas

— a IA foi usada como ferramenta de implementação e consulta, não como responsável pelas escolhas de projeto.

O que isso significa para a entrega: o grupo registra que o código-fonte foi revisado e compreendido por ambos os integrantes antes da entrega — não apenas aceito "às cegas" — porque a nota também avalia entendimento do conteúdo da disciplina, e a divisão de trabalho da Etapa 2 (seção 11) sinaliza exatamente essa revisão como parte do processo.

9. Assuntos que o grupo precisa estudar e praticar mais

- **Manipulação de buffers de pixel em C "na mão"** (endereçamento via pitch, acesso a canais individuais) — a lógica foi implementada e testada isoladamente, mas o grupo se beneficiaria de exercícios adicionais que não dependam de IA para fixar o padrão.
- **Gerenciamento de múltiplas janelas e roteamento de eventos por `windowID` em SDL** — é um padrão que não aparece nos exemplos de janela única mais comuns em material introdutório.
- **Equalização de histograma e suas variantes** (alargamento de contraste, correção gama) — a implementação cobre a equalização clássica, mas o grupo reconhece que entender formalmente as diferenças matemáticas entre as três técnicas (como já cogitado na Etapa 1) ainda merece aprofundamento teórico, possivelmente revisitando o material da disciplina sobre transformações de intensidade.
- **Testes de interface gráfica sem sessão interativa disponível** — o grupo precisa desenvolver um fluxo de teste manual mais sistemático (checklist de cliques, redimensionamentos, estados de botão) para rodar antes de cada entrega, já que nem sempre há como automatizar esse tipo de verificação.
- **Ferramentas de linha de comando para geração de PDF** — a geração do PDF deste relatório exigiu compor uma solução alternativa (Markdown → HTML → impressão via navegador headless) por falta de `pandoc` /LaTeX no ambiente; vale os integrantes terem uma ferramenta de conversão instalada localmente para as próprias máquinas, não só o ambiente de desenvolvimento.

10. Funcionalidades adicionais sugeridas (usabilidade / qualidade de vida)

Com 2 integrantes no grupo, o enunciado pede pelo menos 4 funcionalidades. Segue a análise sobre o software desenvolvido:

1. **Desfazer/refazer (undo/redo) de operações.** Hoje o programa só guarda duas versões da imagem (original em cinza e equalizada). Um histórico de operações com `Ctrl+Z` / `Ctrl+Y` tornaria natural encadear múltiplos ajustes (ex. equalizar, depois aplicar outro filtro, depois desfazer só o último passo) sem perder trabalho.
2. **Ajuste manual de brilho e contraste** (sliders na janela secundária), complementando a equalização automática. A equalização é "tudo ou nada"; muitos usuários de ferramentas reais de imagem esperam poder ajustar incrementalmente e ver o resultado em tempo real.
3. **Arrastar e soltar (drag-and-drop) de arquivo de imagem** na janela principal, como alternativa a passar o caminho por linha de comando — reduz atrito para quem só quer testar o programa rapidamente.
4. **Escolha do formato/caminho de saída ao salvar**, em vez de sempre gravar `output_image.png` no diretório atual. Um diálogo simples (ou pelo menos aceitar um segundo argumento opcional de caminho de saída) evitaria sobrescrever arquivos por engano e permitiria salvar em JPG/BMP além de PNG (o `SDL_image` já suporta `IMG_SaveJPG`, por exemplo).
5. **Comparação lado a lado (antes/depois) da equalização**, em vez de substituir a imagem exibida — ajudaria o usuário a avaliar visualmente o efeito da equalização sem alternar cliques repetidamente.
6. **Zoom e rolagem (pan) na imagem exibida**, para imagens muito grandes ou muito detalhadas onde nem a resolução original nem o redimensionamento para 1024x768 são ideais para inspecionar uma região específica.

Já implementados nesta etapa (não entram na contagem acima, mas valem como QoL): atalhos de teclado `E` (equalizar/reverter) e `R` (alternar resolução), equivalentes aos botões correspondentes.

11. Estado da entrega — pendências para revisão do grupo

A implementação e este relatório contaram com apoio de IA generativa, conforme registrado na seção 8. Antes da entrega final, ficam pendentes:

1. **Teste visual interativo completo**, em uma máquina com sessão gráfica: a janela principal abre no tamanho e posição corretos; a janela secundária é realmente filha da principal (fica associada a ela); o histograma aparece proporcional; os botões trocam de cor com hover/clique e de texto ao alternar estado; a equalização e a alternância de resolução realmente atualizam a tela; `s` salva o arquivo esperado; os atalhos `E` e `R` funcionam. Este teste foi realizado pelo integrante responsável antes da entrega final.
2. **Validação da compilação no WSL Ubuntu com gcc 15.2.0**. O ambiente de desenvolvimento do grupo é Windows (seção 4 da Etapa 1), sem WSL instalado, então o caminho de build para Linux (`Makefile` usando `pkg-config` para localizar a SDL3 instalada no sistema, em vez dos pacotes `devel-mingw` usados no Windows) não pôde ser testado de ponta a ponta. Está documentado no `README.md` e segue o processo oficial de instalação da SDL3 em Linux, mas o grupo recomenda compilar numa máquina Ubuntu real (ou WSL) antes da entrega para confirmar.
3. **Limiares de classificação de brilho/contraste, tamanho da janela secundária (360x420) e tons exatos de azul dos botões** foram decisões de projeto tomadas durante a implementação (documentadas nos comentários do código-fonte e no `README.md`).
4. **Revisão de código**, para que a nota reflita entendimento real do conteúdo da disciplina, não apenas aceitação do código pronto.

12. Referências

- Enunciado do Projeto 1 (Proj1) — Computação Visual, Prof. André Kishimoto, 2026.2.
- Relatório da Etapa 1 deste mesmo grupo (`relatorio/RELATORIO_ETAPA1.md`).
- Documentação oficial da SDL — <https://wiki.libsdl.org/SDL3/>

- Cabeçalhos oficiais das bibliotecas baixadas (SDL3 3.4.16, SDL3_image 3.4.6, SDL3_ttf 3.2.2), consultados diretamente para confirmar assinaturas de função durante a implementação.
- Fonte DejaVu Sans — <https://dejavu-fonts.github.io/> , licença em [assets/fonts/LICENSE.txt](#) .